

# **EDDIE MAY NCUBE CAPSTONE PROJECT WRITEUP**

## **STAGE 1: INGESTION**

### **STEP 1: SETUP OF MY PROJECT FOLDER ON VS CODE CREATING FILES AND GITIGNORE/ENV... GENERATION OF THE TICKERS.**

- I first chose my 6 tickers which where
  - AAPL... (APPLE)
  - MSFT... (MICROSOFT)
  - NVDA...(NVIDIA)
  - AMZN...(AMAZON)
  - META... (META)
  - GOOGL... (GOOGLE)
- Then went on Yahoo finance to check the trades of each of the tickers
- I then generated a fix\_prices.py file script and a trade\_log\_csv to generate a trades sheet that I will use on google sheet placed them on a folder tickers .
- Installed packages like (pip install yfinance pandas), and then ran the (python fix\_prices.py) to generate a real trades log csv .
- Once generated I then created a new google sheet spreadsheet then imported the data from the real trades log csv to trades

### **STEP 2: GITHUB ACTIONS**

- Created a github repository {  
[https://github.com/mayeddiencube02-boop/Eddie\\_Ncube\\_portfoliopulse](https://github.com/mayeddiencube02-boop/Eddie_Ncube_portfoliopulse)}
- Checked if I installed git version , then did my git commands which are
  - git init
  - git branch -M main
  - git add .
  - git status,
  - git commit -m "Initial commit: trade log and price script"
  - git remote add origin [https://github.com/mayeddiencube02-boop/Eddie\\_Ncube\\_portfoliopulse](https://github.com/mayeddiencube02-boop/Eddie_Ncube_portfoliopulse).git
  - git push -u origin main

### **STEP 3: SETTING UP NEON (Database)**

- Created a new project called Eddie\_Ncube\_portfoliopulse
- Connected the project in order to copy the connection string which I therefore went and pasted it in my .env file in vs code
- Created a testing connection python file named (scripts\test\_connection.py) which had packages to connect with my neon postgres
- Then ran my tests( python scripts\test\_connection.py) after some debugging errors then it successfully connected

#### STEP 4: GENERATION OF THE SYNTHETIC DATA FILE READY TO LOAD ON POSTEGRESS

- Using our recent seed\_data file , I had to compare and generate a synthetic file and named it load\_stock\_prices.py
- The file will load stock prices from yfinance into neon(postgress)
- The file will create a schema (raw) and table called raw\_stock\_prices
- The file will run from start day and later on will ask for updated data ,it will download only the last 10 days rather getting all the information
- Upeserting the rows was done to "update if it exists, insert if it doesn't. for the rows that would have been inserted

#### STEP 4: CHECK IF THE PYTHON FILE WAS SUCCESSFULLY LOADED INTO NEON DATABASE

- **In my sql editor I ran the following queries**

```
SELECT ticker,  
COUNT(*), MIN(price_date), MAX(price_date)  
FROM raw.stock_prices  
GROUP BY ticker;
```

- This showed all my 6 tickers with the minimum price date and maximum price date
- I went on to run queries for each ticker checking their price date comparing with the actually yahoo finance

-

**-- TO CHECK FOR APPLE (AAPL) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'AAPL'  
AND price_date in ('2026-04-07', '2026-05-12', '2026-06-24', '2026-08-19')  
ORDER BY price_date;
```

**-- TO CHECK FOR MICROSOFT(MSFT) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'MSFT'  
AND price_date IN ('2026-04-07', '2026-05-27', '2026-07-15')  
ORDER BY price_date;
```

**-- TO CHECK FOR NVIDIA (NVDA) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'NVDA'  
AND price_date IN ('2026-04-14', '2026-05-19', '2026-07-08', '2026-09-02')  
ORDER BY price_date;
```

**-- TO CHECK FOR AMAZON(AMZN) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'AMZN'  
AND price_date IN ('2026-04-21', '2026-06-03', '2026-07-22')  
ORDER BY price_date;
```

**-- TO CHECK FOR META (META) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'META'  
AND price_date IN ('2026-04-28', '2026-06-10', '2026-08-05')  
ORDER BY price_date;
```

**-- TO CHECK FOR GOOGLE (GOOGL) PRICE DATE**

```
SELECT ticker, price_date, CLOSE  
FROM raw.stock_prices  
WHERE ticker = 'GOOGL'  
AND price_date IN ('2026-05-05', '2026-06-17', '2026-08-12')  
ORDER BY price_date;
```

- Finally I had to push the connection using the command  
**git add scripts**  
**git commit -m "Add yfinance loader and connection test"**  
**git push**
- the loader and connection successfully loaded on github

#### STEP 5 : COONECTING AIRBYTE AND NEON

– The aim is to bring the other 2 sources

- FIRST created a destination called (portfoliopulse) with my neon login details in it
- SECOND created a source (That is my google sheet/portfoliopulse ) as my primary key as
- This created the connection between AIRBYTE AND NEON
- The trades sheet was successfully loaded into neon , ran queries like  
**SELECT \***  
**FROM raw.trades;**
- This displayed all the 20 rows airbyte id

#### STEP 6: EXCHANGE RATE CONNECTION (BUILDER CONNECTION)

- I created a stream connector that will ask airbyte from the naijashop currenecy exchange api which I named it ( exchange-usd-ng)
- I then used a free api from open sources (fawazahmed0 currency API.)
- .First, the address to call, with a placeholder where the date goes.
- Second, how the dates work: start on 2026-04-01, move forward one day at a time, and remember the last date fetched so the next run only asks for new days. That is what "incremental sync" means.
- Third, which field identifies each record, which is date
- The connector is now ready to publish. After that, Airbyte can run it once per day, one request per day, and load the results into your Neon database

#### STEP 7: CONNECTING {api-usd-ngn-rates} to NEON

- Created a new source from the custom sources called {api-usd-ngn-rates}
- Connected it with my destination which is the portfoliopulse that I created before
- Then successfully created a connection and synced my data ( About .....rows loaded)
- CHECKING ON NEON
- I used the following query
- FIRST CHECKED MY TABLE SCHEMA

```
SELECT table_schema, table_name  
FROM information_schema.tables  
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')  
ORDER BY table_schema, table_name;
```

Output : raw.api\_usd\_ngn\_rates and stock\_prices and trades

- Then followed by

```
SELECT  
COUNT(*) AS total_rows,  
COUNT(distinct date) AS distinct_days,  
MIN(date) AS first_day,  
MAX(date) AS last_day  
FROM raw.api_usd_ngn_rates;
```

Output: pulled out 190 rows

- Then Pulled out the naira rate to see it actually confirms

```
SELECT d::date as missing_date  
FROM generate_series('2026-04-01'::date, current_date - 1, interval '1 day') d  
WHERE d::date not in (select date::date from raw.exchange_rates)  
ORDER BY 1;
```

Output: Statement executed successfully

**NB: INGESTION WAS SUCCESSFULLY DONE //**

## **STAGE 2 : TRANSFORMATION USING DBT**

Importance of dbt is to turn the three raw tables into the report

- We will first create my staging files which are { stg\_stock\_prices, stg\_trades and stg\_api\_usd\_ngn\_rates)
- We will also create a mart named { mart\_portfolio\_value} for each day and ticker, shares held, market value, cost basis and gain/loss all in both USD and NGN.
- Days with no exchange rate will use the most recent earlier rate
- Lastly will tests : unique and not\_null on the keys.

STEP 1: Installed dbt in my terminal

- Created the stage files
  1. Stg\_exchange\_rates.sql
  2. Stg\_stock\_prices.sql
  3. Stg\_trades.sql.sql
- Created the marts files
  1. Mart\_portfolio\_daily.sql
  2. Mart\_portfolio\_values.sql
- Created tests
  1. Assert\_trade\_prices\_match\_market\_close.sql

STEP 2: GITHUB ACTIONS

- After successfully transforming my files I had to run my git commands using the following

```
cd ..
git add .
git commit -m "Add dbt project"
git push
```

**NB: THIS THEN RAN SUCCESSFULLY AND MY DBT PROJECT WAS ON GITHUB//**

## **STAGE 3: ORCHESTRATION AND AUTOMATION**

### **STEP 1: GITHUB WORKFLOW**

- Created files which are { .github\workflows\daily\_pipeline.yml and scripts\trigger\_airbyte.py }

#### **HOW THE WORKFLOW RUNS**

1. load\_stock\_prices.py refreshes the prices.
  2. trigger\_airbyte.py starts both Airbyte syncs and waits for them to finish.
  3. dbt run.
  4. dbt test.
- Successfully created my daily pipeline yml
  - Next was to add secrets and variables using NEON AND AIRBYTE Details
  - In AIRBYTE I First created a connection and named it portfoliopulse-workflow in order to generate client id and client secret
  - Created the following secrets
    1. AIRBYTE\_CLIENT\_ID
    2. AIRBYTE\_SECRET\_ID
    3. AIRBYTE\_CONNECTIONS\_ID
    4. NEON\_DATABASE\_URL
    5. NEON\_DB
    6. NEON\_USER
    7. NEON\_HOST
    8. NEON\_PASSWORD
  - Took all the information from my airbyte and neon database url
  - Had a lot of debugging done here as my workflow could not run , AFTER 5 DEBUGS the workflow ran

- CREATED A ROLE FOR READ ONLY END USER – with the following details



```
CREATE ROLE reviewer login PASSWORD 'portfoliopulse';  
GRANT CONNECT ON DATABASE neondb TO reviewer;  
GRANT USAGE ON SCHEMA analytics, raw TO reviewer;  
GRANT SELECT ON ALL TABLES IN SCHEMA analytics, raw TO  
reviewer;  
ALTER DEFAULT PRIVILEGES FOR ROLE neondb_owner IN  
SCHEMA analytics GRANT SELECT ON TABLES TO reviewer;  
ALTER DEFAULT PRIVILEGES FOR ROLE neondb_owner IN  
SCHEMA raw GRANT SELECT ON TABLES TO reviewer;
```

NB: FINALLY THE WORKFLOW IS RUNNING //



## **STAGE 4: VISUALIZATION { METABASE VIA DOCKER }**

### STEP 1: INSTALLATION OF METABASE

- First installed the metabase library in the docker terminal

**docker run -d -p 3000:3000 --name metabase -v metabase-data:/metabase-data -e MB\_DB\_FILE=/metabase-data/metabase.db metabase/metabase**

- Then I logged in the metabase using the local host <http://localhost:3000> and created my admin account
- Then on my metabase

- CREATED FOUR CHARTS NAMELY
  1. Portfolio value over time (line): with the query

```
SELECT price_date, total_market_value_usd  
FROM analytics.mart_portfolio_daily  
ORDER BY price_date;
```

2. Gain/loss by position (bar), latest day: with the query

```
SELECT ticker, unrealized_gain_loss_usd  
FROM analytics.mart_portfolio_value  
WHERE price_date = (select max(price_date)  
FROM analytics.mart_portfolio_value)  
ORDER BY unrealized_gain_loss_usd desc;
```

3. Current allocation (pie or donut): with the query

```
SELECT ticker, market_value_usd  
FROM analytics.mart_portfolio_value  
WHERE price_date = (select max(price_date)  
FROM analytics.mart_portfolio_value);
```

4. NGN vs USD portfolio value (side by side): Metabase can't put NGN and USD on one axis because the scales differ by about 1,400×, so I had to show two lines on a dual axis, or show two charts next to each other: with the query

```
SELECT price_date, total_market_value_usd, total_market_value_ngn  
FROM analytics.mart_portfolio_daily  
ORDER BY price_date;
```

**NB: ALL MY QUERIES RAN SUCCESSFUL//**

**STAGE 5: CREATION OF ERD DIAGRAM USING DBDIAGRAM.OI AND README  
FILE**

- Used dbdiagram.oi to draft the ERD Diagram
- Will use this as the writeup

## **SUMMARY**

### **PortfolioPulse: Design Write-up**

#### **Goal**

Answer one question automatically every day: what is my investment portfolio worth, in USD and in Naira, over time? The portfolio is six stocks (AAPL, MSFT, NVDA, AMZN, META, GOOGL) bought across 20 logged trades.

#### **Architecture**

Three sources feed one Neon Postgres database. Stock prices come from yfinance through a Python loader. The trade log lives in a Google Sheet and the USD→NGN exchange rate comes from a public currency API; both are loaded by Airbyte. Everything lands in a raw schema, dbt Core transforms it into an analytics schema, and Metabase (in Docker) reads the analytics tables. One GitHub Actions workflow runs every weekday: refresh prices, trigger both Airbyte syncs through its API, then dbt run and dbt test. Any failing step fails the run.

#### **Data model**

raw.stock\_prices (key: ticker + date), raw.trades (key: trade\_id), raw.api\_usd\_ngn\_rates (key: date).

Staging views clean and type the raw data (trades arrive as text from the sheet).

mart\_portfolio\_value: one row per ticker per day with shares held, average cost, market value and unrealized gain/loss in USD and NGN.

mart\_portfolio\_daily: portfolio totals per day, feeding the line and side-by-side charts.

#### **Key decisions**

Upserts (INSERT ... ON CONFLICT DO UPDATE) make every load idempotent, so retries and re-runs cannot create duplicates.

Exchange rates have gaps, so each day uses the latest rate on or before it; the NGN value is never null. Gain/loss in NGN is the USD gain/loss times that day's rate.

The originally planned rate API went offline (HTTP 402). I replaced it with a custom Airbyte connector on a free public API, incremental by date, with a handler that ignores 404 for days with no file.

Trade prices were checked against real closing prices; a dbt test warns if a logged price differs from that day's close by more than one cent.

### **Quality and reliability**

27 dbt tests cover unique and not-null keys, allowed tickers, a non-null NGN rate and the trade-price check. The daily workflow stops on the first failure, so a bad day shows as a red run instead of silently wrong numbers. Reviewers get a read-only database role.

### **Limitations and next steps**

Prices are daily closes only; no intraday values.

Cost basis uses the average price paid; there are no sells, fees or dividends.

Metabase runs locally in Docker; hosting it would make the dashboard permanently available.